

Analysis of 'When Malware is Packin' Heat; Limits of Machine Learning Classifiers Based on Static Analysis Features'

Thomas Turner
M14507785

Nathan Hafely
enter M number here

1 INTRODUCTION

With the rise of machine learning and artificial intelligence in recent years, many malware analysis experts have begun to research how these new tools can be used in their field. Many papers have begun to be published on the topic of using machine learning classifiers to distinguish between packed benign and packed malicious samples. The paper that we analyzed aims to verify the accuracy of these tests and see for themselves if machine learning can indeed be used to distinguish the maliciousness of packed samples based on static features alone [1].

Creating tools to automatically detect if a sample is malicious or not is very sought after in the malware field. Currently the best and most effective way to determine if a sample is malicious is to have an expert analyzing it using various static and dynamic analysis tools. This process entails placing the malware in an isolated environment and analyzing the static features such as imports, strings, and assembly code without running the code. Then the analyst will run debug the code to see any registry key changes, file system changes, or network connections the sample is attempting to make. This process can be very tedious and can take an expert a long time to complete.

There are also many online tools now such as VirusTotal that be used to automatically scan a sample to determine if it is malicious or not. But, this method also has many downsides. VirusTotal works by using signature-based detection methods to determine if a sample is malicious. So, if the signature of a sample is similar to a known malware, then VirusTotal will detect that the given sample is potentially malicious [1]. This technique is effective at detecting previously known malware, but will struggle on types of malware that it has never encountered before.

VirusTotal is also known to struggle with packed samples. The paper mentions a study that was conducted by Lakshman Nataraj. In this study, they collected a large number of benign Windows executables and then packed them with with four different common packers. Next, they submitted all of these samples to VirusTotal. From this experimnt, they found that 96.7 percent of the samples caused ten or more malicious detections on VirusTotal [1]. This shows that VirusTotal also uses packing as a strong indicator of maliciousness instead of the actual functionality of a sample.

Nowadays, many benign samples are also packed to protect the authors from getting their intellectual property stole [1]. Therefore, it is no longer the normal for exclusively malicious samples to be

packed. Figure 1 below shows the results of a brief experiment that the researchers conducted. In this experimnt, they collected a large number of samples from the real world and then analyzed them to see how many of them were packed based on each category of benign, malicious, and suspicious [1]. The results show that in the worst quarters around 50% of the benign samples that they collected were packed [1].

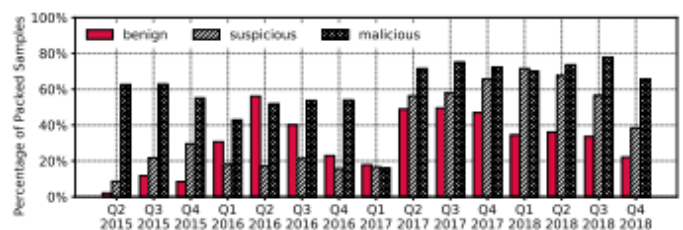


Figure 1: Prevalence of packed samples in the wild.

Through their experiments, the researchers show that packing routines do preserve some information on the function and maliciousness of a sample, but it is not enough to be applied to packers and packing routines that the classifier has never seen. Additionally, the classifier is susceptible to adversarial attacks [1]. This means that if malware authors can place benign code in their malware to try and confuse the classifier into thinking that the entire program is benign.

2 SYSTEM/THREAT MODEL

For their experiments, the researchers used samples exclusively from the Windows x86 operating system. The samples can be broken up into four unique categories: unpacked benign, packed benign, unpacked malicious, and packed malicious [1]. For each individual experiment, the data was broken up into a training set and a testing set. The type of classifier that was used for the analysis was a random forest model. Additionally, the classifier was trained on nine different sets of features in the samples (PE Headers, PE Sections, DLL Imports, API Imports, Rich Header, Byte n-grams, Opcode n-grams, Strings, File Generic) [1].

3 METHODOLOGY

For the researchers to be able to accurately train a machine learning classifier to distinguish between a benign or malicious sample, they need lots of accurate data and independent features of each piece of data. The researchers also broke up the experiments into four different categories based upon what research question the results were answering.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ACM XXXX'18, XX 2018, XX, XX, XX
© 2018 Copyright held by the owner/author(s).
ACM ISBN 123-4567-24-567/08/06...\$15.00
https://doi.org/10.475/123_4

3.1 Building the Datasets

The researchers began building the dataset by collecting data from a commercial anti-malware vendor and a labeled dataset called EMBER. The data from these two sources was then placed into the four categories of unpacked benign, packed benign, unpacked malicious, and packed malicious [1]. From the commercial anti-malware vendor, they collected a total of 29,573 executables. All of the samples from this source were pre-labeled, so no further analysis had to be done to place these samples into the correct category. From the EMBER dataset, they collected a total of 800,000 executables. But, this dataset was only labeled between if the sample was benign or malicious. So, 56,411 of the samples were taken and then entered into the commercial anti-malware vendor to further categorize them into either packed or unpacked. As seen below in figure 2, the the data from these two sources totals to 85,984 samples [1].

Samples' Origin	Malicious/Benign Label's Source			# of Samples
	VirusTotal	Comm. Anti-mal.	EMBER	
(1) Comm. Anti-malw.	A	A	N/A	29,573
	A	B	N/A	15,736
	B	B	N/A	13,046
	A	M	N/A	13,837
	M	M	N/A	13,536
(2) EMBER	A	A	A	56,411
	A	A	B	24,348
	A	B	B	24,225
	B	B	B	24,223
	A	A	M	32,063
	A	M	M	31,087
	M	M	M	31,066
(1) $\cup$ (2)	A	A	A	85,984
	B	B	B	37,269
	M	M	M	44,602

Figure 2: Raw dataset collected from the commercial anti-malware vendor and the EMBER dataset.

This data was then used to build the Wild Dataset. As the name suggests, this dataset is meant to be a representation of samples in the real world. This dataset was built by verifying all of the raw data that was collected to ensure that the labels were correct. To determine if the benign/malicious labels were correct, the researchers cross-referenced all of the samples to between VirusTotal, the anti-malware vendor, and the EMBER dataset. If there were any discrepancies between these three sources, then those samples were discarded. Using this method, a total of 4,113 samples were discarded [1]. To verify that the packed/unpacked labels were accurate, the researchers compared all of the samples against the anti-malware vendor, Deep Packer Inspector (dpi), and various other malware tools (as can be seen in Figure 3). From this analysis, 31,147 samples were discarded [1]. This narrowed down wild dataset to a total of 50,724 samples.

Next, the researchers built a separate dataset called Lab Dataset. This dataset consisted of all of the samples in the Wild Dataset, but packed by nine different known packers. Some of the samples were not able to be packed by all of the packers, so those samples were discarded [1]. This resulted in the Lab Dataset having a total of 341,444 samples (shown in Figure 4) [1].

Tool	Benign		Malicious	
	packed	unpacked	packed	unpacked
(1) vendor's sandbox	10,463	16,162	26,699	15,095
(2) dpi	6,049	20,576	27,995	13,799
(3) <i>Analyze</i>	1,239	17,436	5,376	19,457
(4) <i>PEiD+F-Prot</i>	1,189	25,436	2,630	39,164
(5) <i>yara</i>	1,524	25,101	3,882	37,912
(6) <i>Exeinfo PE</i>	1,088	25,537	5,770	36,024
(1)+(2)+(3)+(4)+(5)+(6)	12,647	4,396	33,681	5,752

Figure 3: Packers Used to Verify the Packed/Unpacked Labels on Wild Dataset.

Packer	# Benign Samples	# Malicious Samples	Keeps Rich Header?	# Invalid Opcodes
Obsidium	16,940	31,492	29.82%	0
Themida	15,895	26,908	✓	0
PECompact	5,610	28,346	✓	723
Petite	13,638	25,857	✓	318
UPX	9,938	20,620	✓	0
kkrunchy	6,811	15,494	19.68%	61
MPRESS	11,041	11,494	19.84%	629
tElock	5,235	30,049	✓	8
PELock	6,879	8,474	20.60%	461
<i>AES-Encrypter</i>	17,042	33,681	✗	0

Figure 4: Packers Used to Build the Lab Dataset.

3.2 Identifying the Features

To accurately train a classifier to determine between benign or malicious samples, you need good features to train the classifier on. The researchers identified nine different sections of the samples to grab the features from. These sections were PE Headers (28 features), PE Sections (570 features), DLL Imports (4,305 features), API Imports (19,168 features), Rich Header (66 features), Byte n-grams (13,000 features), Opcode n-grams (2,500 features), Strings (16,900 features), and File Generic (small number of features) [1]. All of these sections represent important aspects of the samples that can be used to decipher some of the functionality of the program.

4 EXPERIMENT RESULTS AND ANALYSIS

This is the Experiment Results and Analysis Section.

5 CONCLUSION AND DISCUSSION

This is the Conclusion and Discussion Section.

REFERENCES

- [1] Hojjat Aghakhani and Fabio Gritti and Francesco Mecca and Martina Lindorfer and Stefano Ortolani and Davide Balzarotti and Giovanni Vigna and Christopher Kruegel. 2020. 'When Malware is Packed' Heat: Limits of Machine Learning Classifiers Based on Static Analysis Features. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.